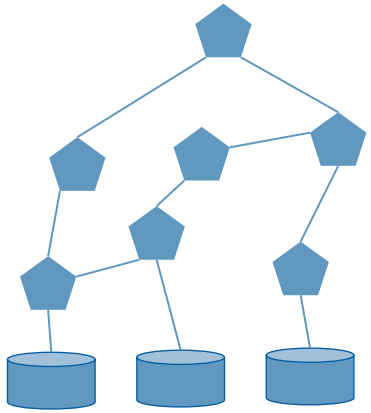


ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών
Τεχνολογίες Υπηρεσιών Λογισμικού

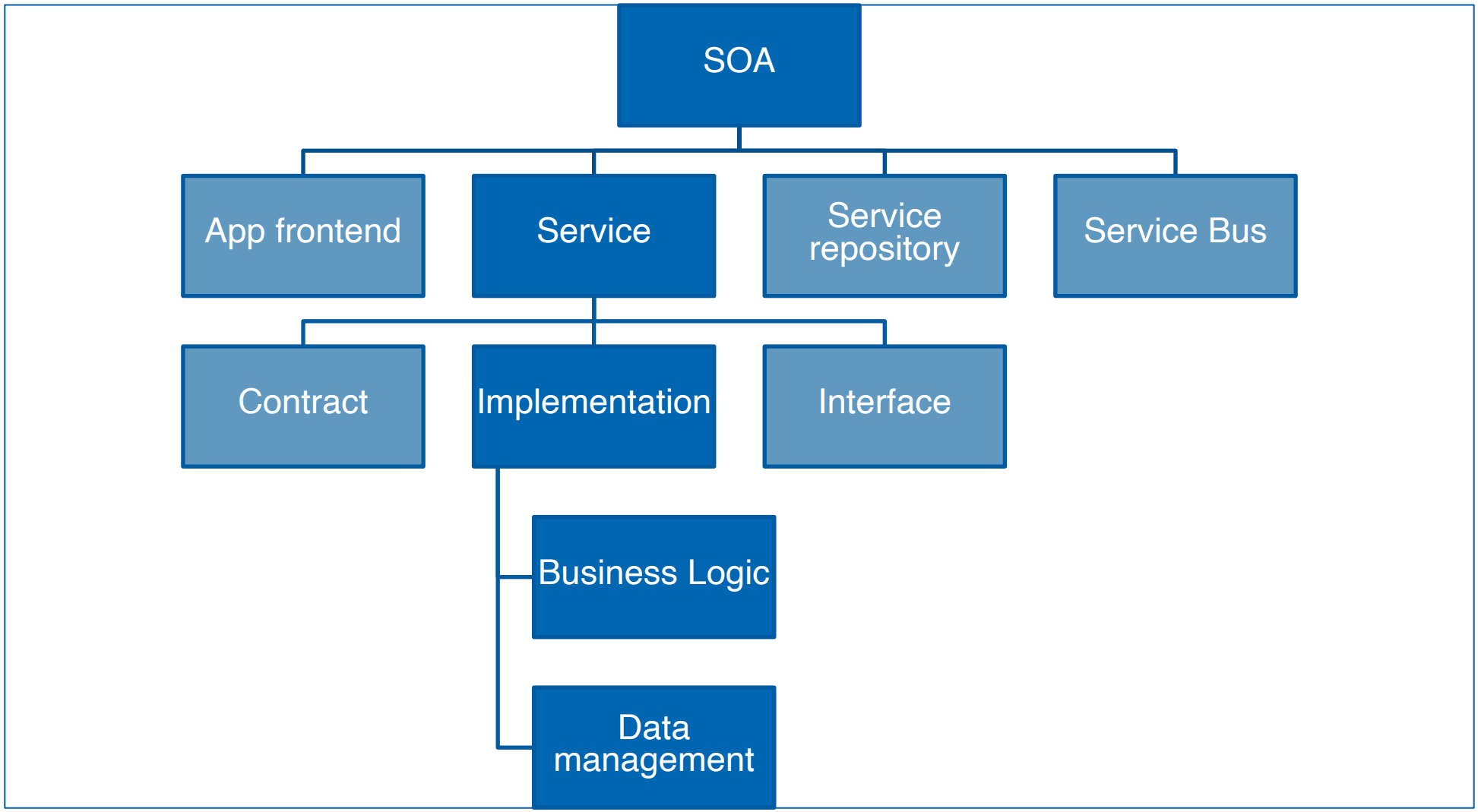
Τεχνολογίες Υπηρεσιών Λογισμικού

Β.Βεσκούκης, Καθ.ΕΜΠ (v.vescoukis@cs.ntua.gr)

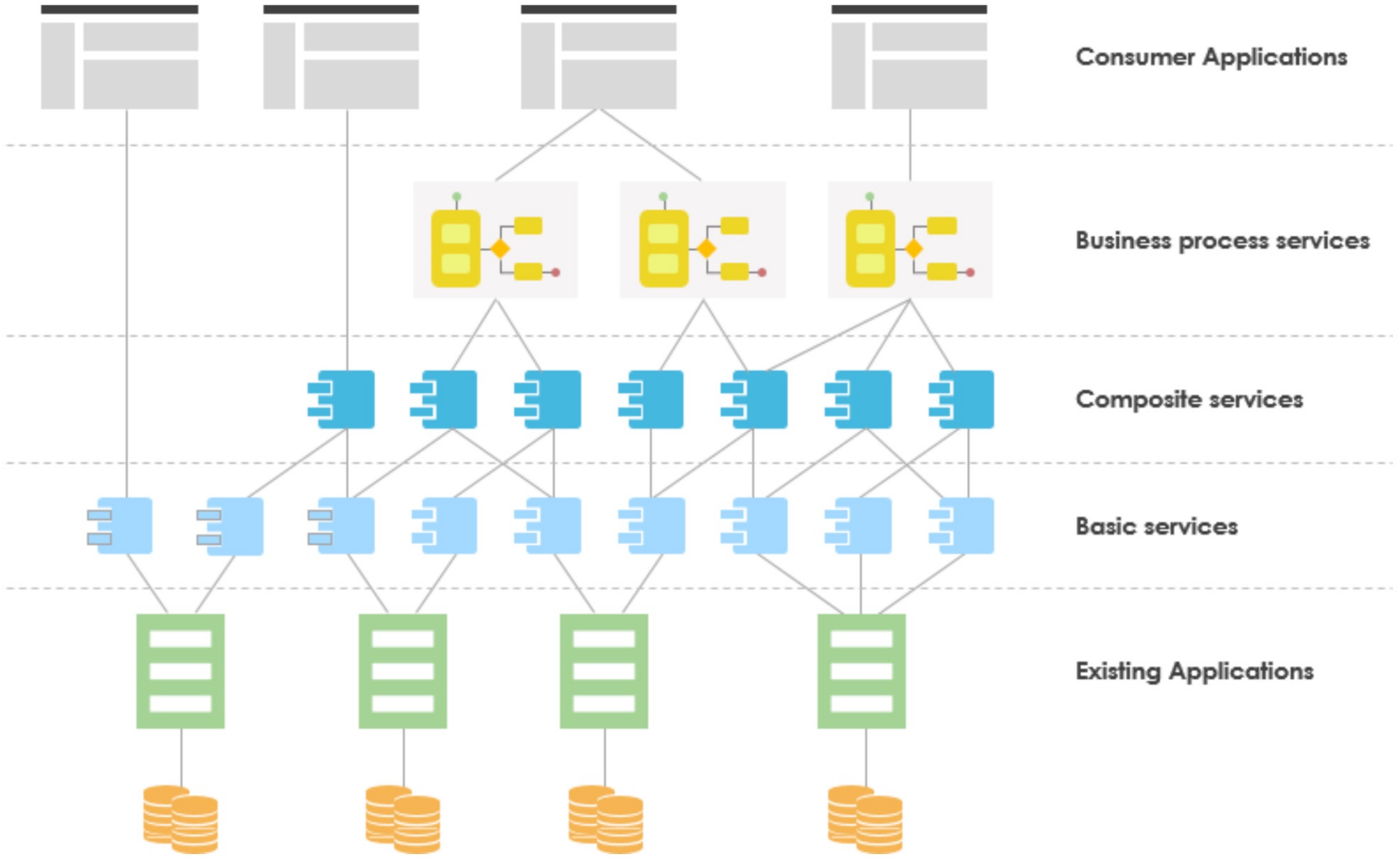


Microservices

Service-Oriented Architecure



SOA: Service-Oriented Architecture



Source: visual-paradigm.com

Microservices

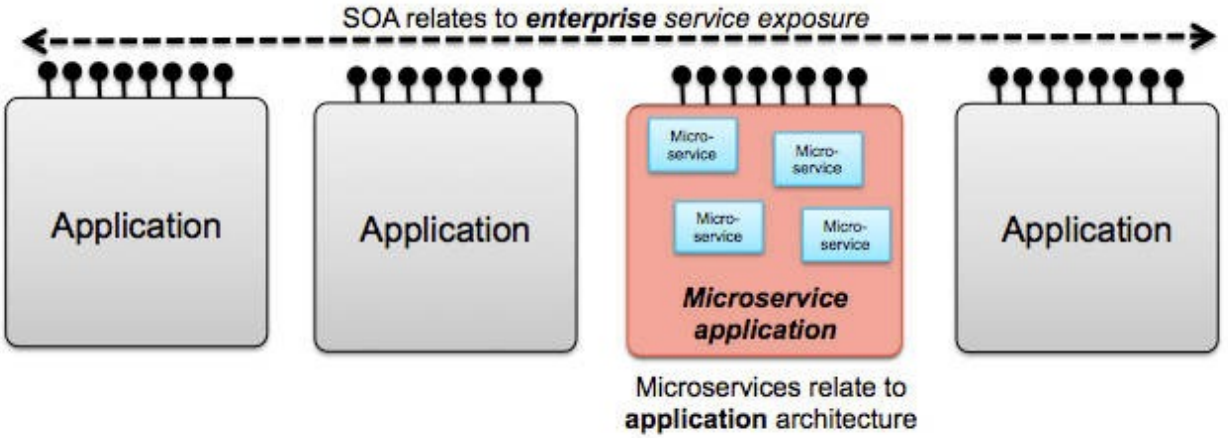
SOA at a different scale

- Same design principles **except** that one microservice may not offer complete business function(s)
- Small size (goes with the name)
- Reusability is not a must (possible code duplication)
- Asynchronous communication (no service bus) enables better fault tollerance
- Data is available *inside* each microservice: possible data duplication, complexity

SOA - microservices

SOA is about…

- Exposing application functions as service interfaces
- Facilitating using data and logic offered by services



Source: IBM

Microservices is about…

- Structuring individual applications from small, independent, agile, scalable and available components

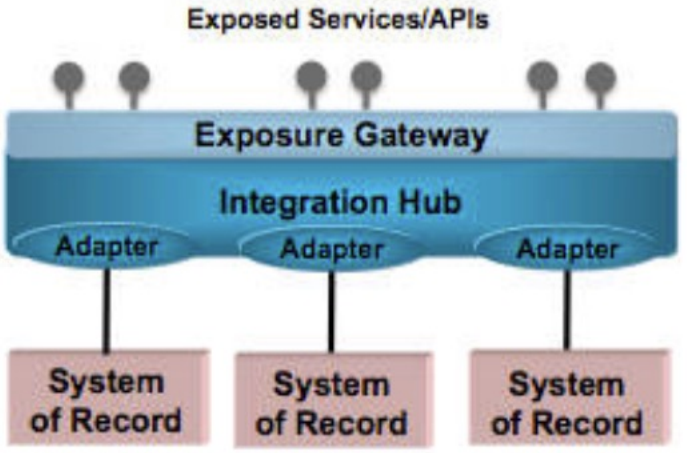
SOA is about integration: technical view

Use standard mechanisms to facilitate re-use of services in new apps

- JSON/HTTP, ~~SOAP/HTTP~~

Focus on integration, not on application design!

The availability of this mechanism in a context is often refferred to as “Enterprise Service Bus”

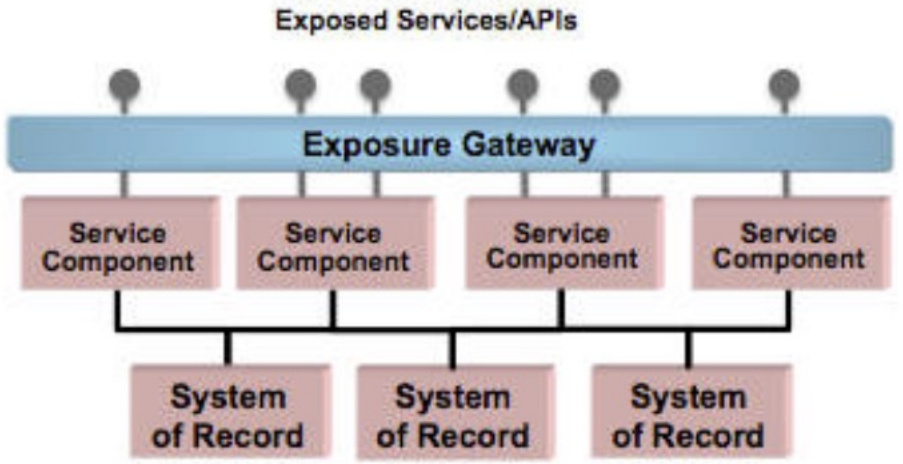


Source: IBM

SOA is about integration: business view

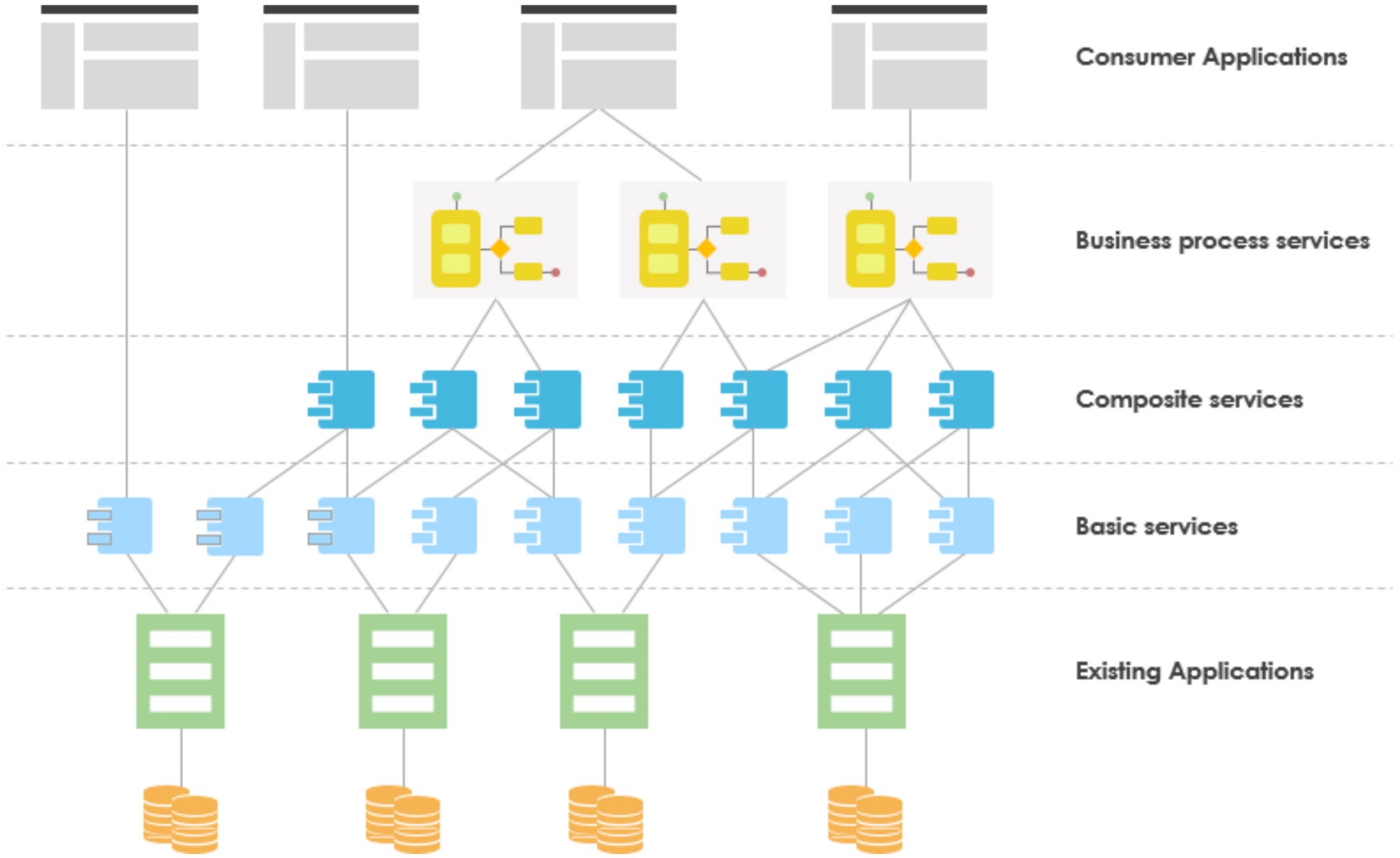
Expose something that “makes sense” to business processes

- Services (should) do something useful that can be re-used in another app
- Use “service components” to achieve this



Source: IBM

SOA: Business functionality from services



Source: visual-paradigm.com

Conception leads structuring

Single responsibility principle

- …every class or module in a program should have responsibility for just a single piece of that program's functionality. Further, the elements of that responsibility should be encapsulated by the responsible class rather than spread out in unrelated classes…
- a class should have only one reason to change

Robert Martin

Microservice architecture

Focus on application architecture

- Break down into small, isolated functions and expose as “services”
- Develop & deploy independently!

Each service has

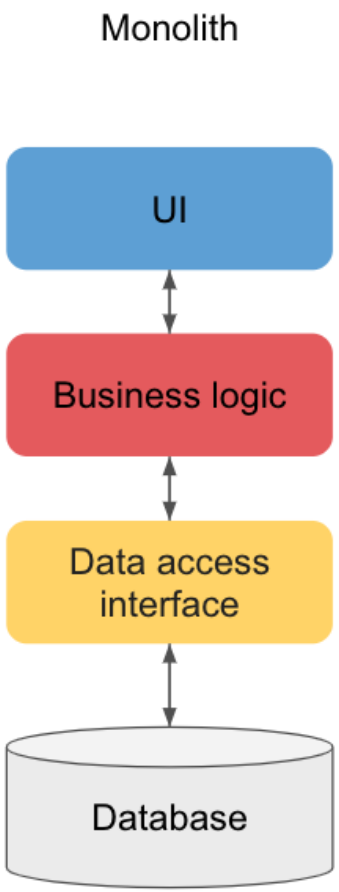
- A single responsibility
- Access to the data it needs
- Limited business functionality
- Limited re-usability
- Maximized agility & resilience

Microservices compared to SOA

Key differences

- **Size**: microservices are smaller, each has a single business responsibility
- **Reusability**: Improve agility and resiliency by minimizing reuse
- **Communication**: Use async communication (no ESB)
- **Data duplication**: to provide access to the data a microservice needs, duplication (and complexity) may occur

Monoliths, SOA & microservices

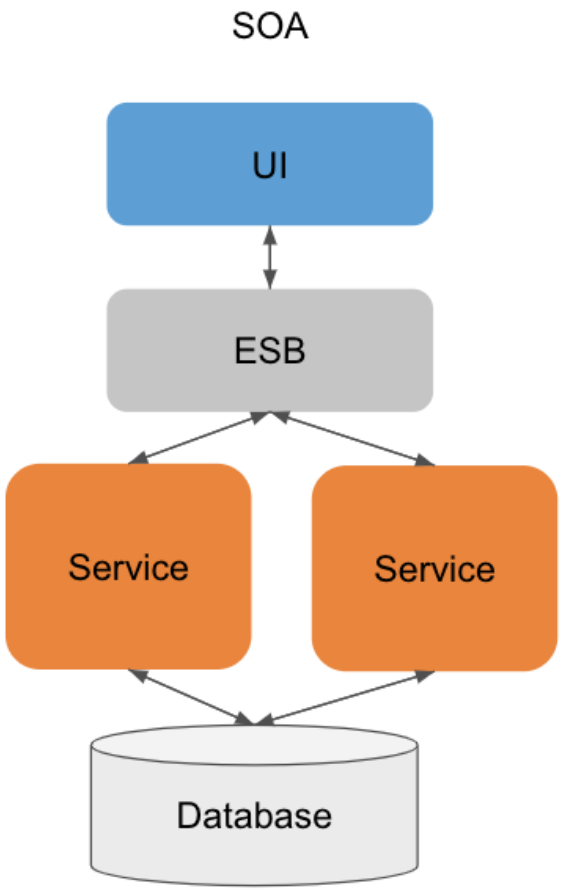


Monolith (generic case)

- **Data is centralized**: one "big" database with per case optimizations, replication, access objects, etc
- **Business logic**: One layer contains all. Access to data is only possible from here.
- **Frontend**: Interactions with users/external systems. No direct access to data – can only call services in "business logic".

Source: Microservices Architecture From A to Z, S.Bouttier @Medium

Monoliths, SOA & microservices

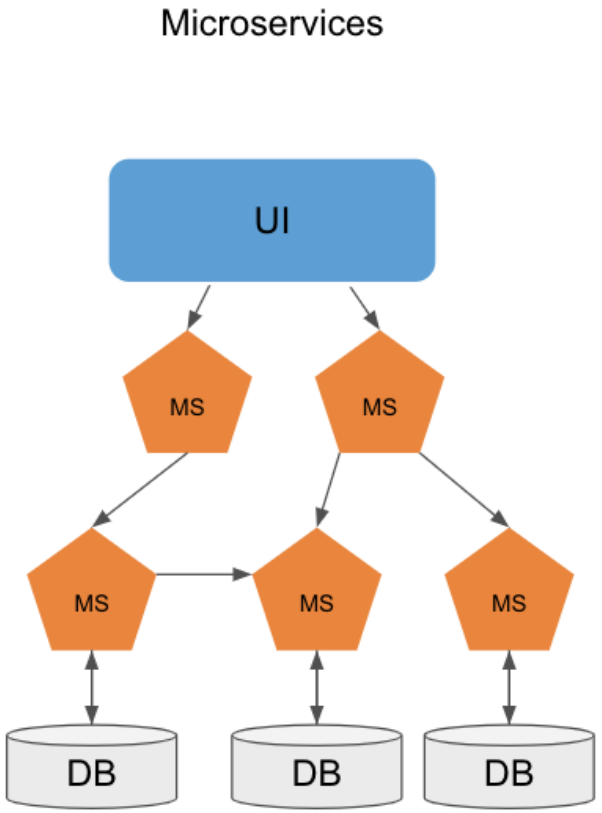


SOA (generic case)

- **Data may be centralized**: each app maintains its own data, in an organization-wide or app-wide database
- **Apps expose services**: Everything is offered as a registered service with specific call/management mechanisms
- **ESB**: All communications occur on a common bus (service)
- **Frontend:** Each app may have its own + new frontend(s) may compose SOA services

Source: Microservices Architecture From A to Z, S.Bouttier @Medium

Monoliths, SOA & microservices

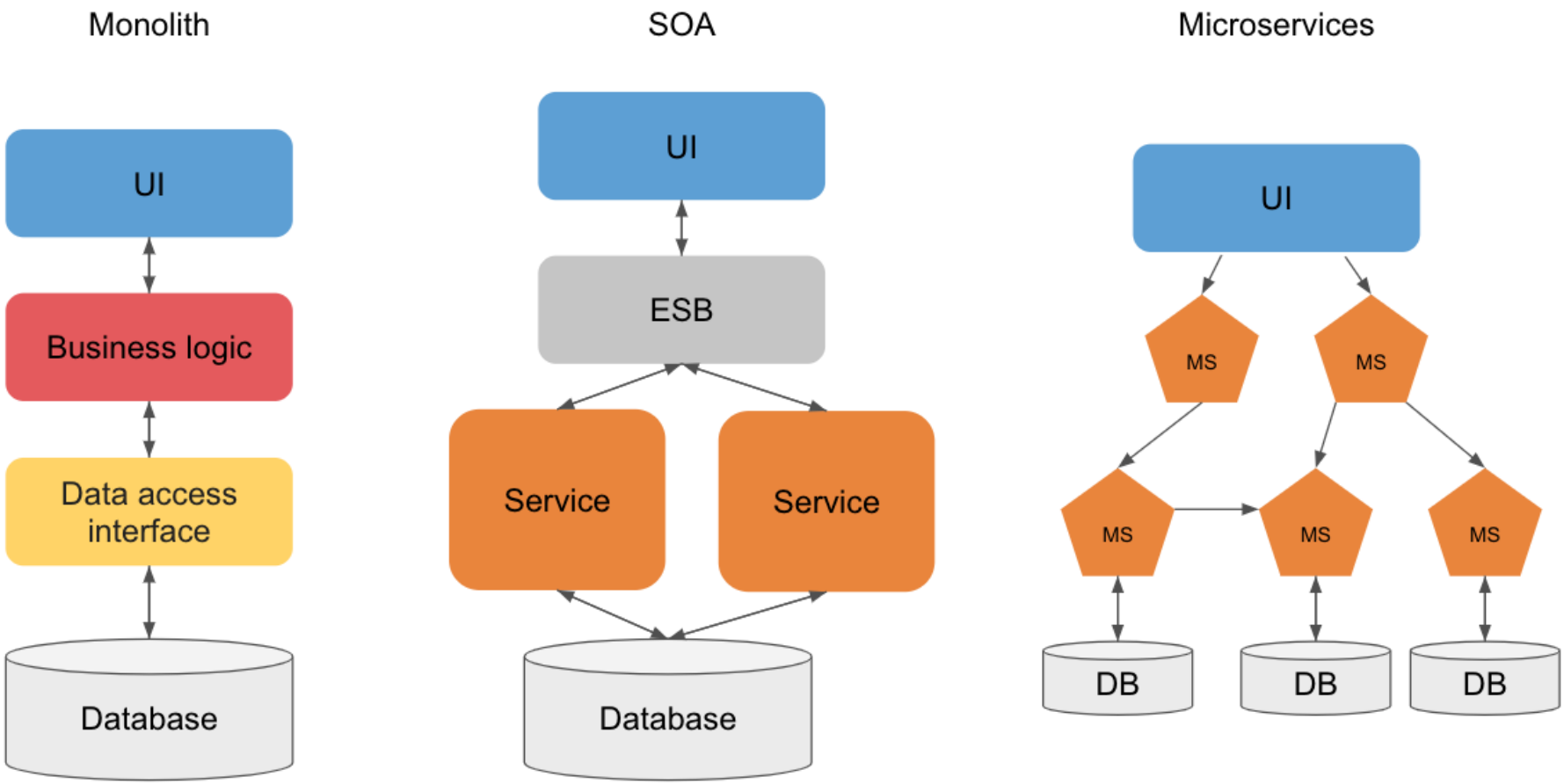


Microservices: Each service has…

- A single responsibility
- Access to the data it needs
- Limited business functionality
- Limited re-usability
- Maximized agility & resilience

Source: Microservices Architecture From A to Z, S.Bouttier @Medium

Monoliths, SOA & microservices



Source: Microservices Architecture From A to Z, S.Bouttier @Medium

Microservices

Pros

- Independend development, deployment, scaling
- Small teams, no sharing of source code, simpler code, data isolation (many databases)
- Resilience by reducing impact when a service fails
- Easier to implement in cloud ecosystems

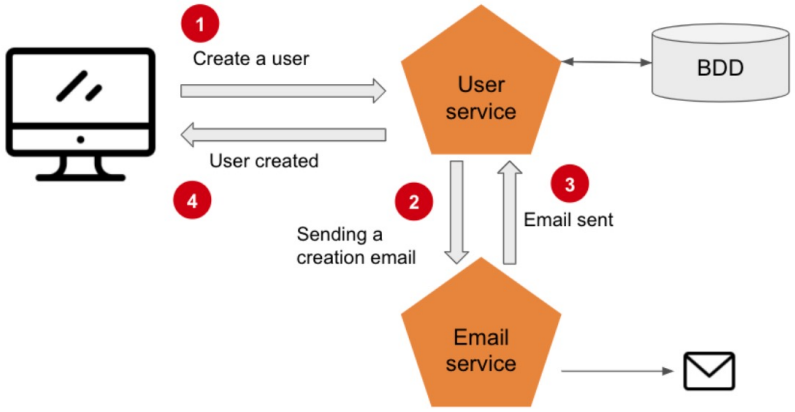
Cons

- System complexity, deployment management
- Testing becomes more complicated
- Distributed databases have data integrity challenges, especially for transaction-based applications
- Network latency, critical for complex communications between services

No universal rule, choice depends on the context!

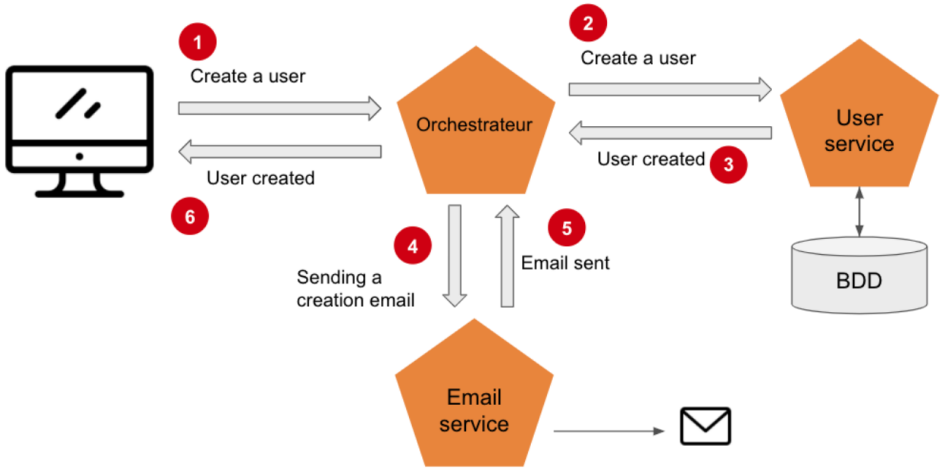
Microservices communication

Use of REST calls vs...



- Dependencies become complex
- Complex is difficult to maintain
- Behaviour is impacted
- Can do better

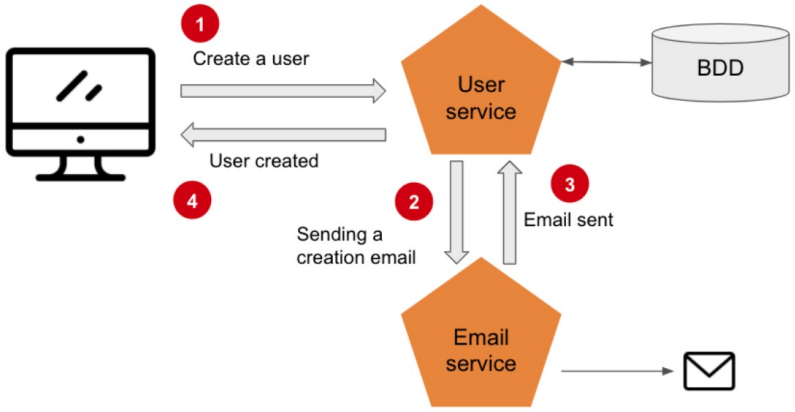
1: orchestration



- Add a service to manage dependencies ("orchestrator")
- The orchestrator should not fail

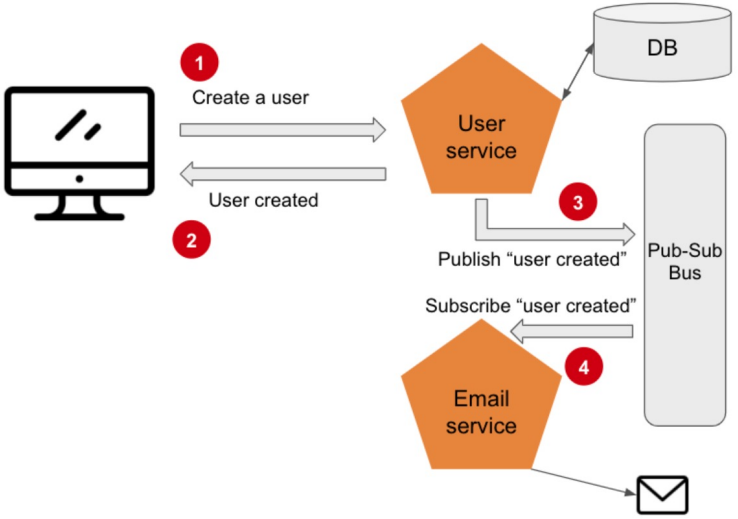
Microservices communication

Use of REST calls vs...



- Dependencies become complex
- Complex is difficult to maintain
- Behaviour is impacted
- Can do better

2: choreography



- Use a publish-subscribe model (no direct or mediated communication)
- Data consistency concerns

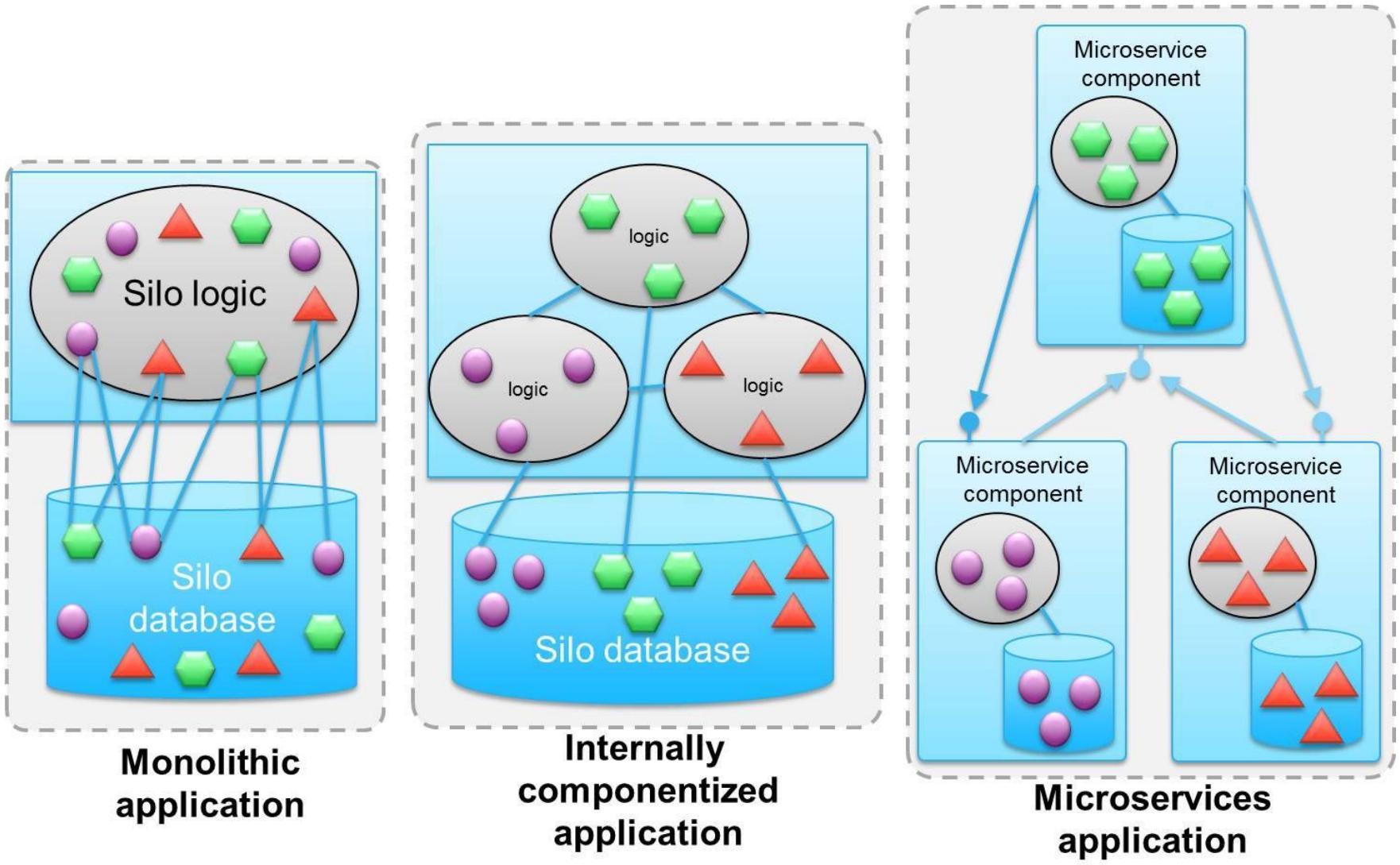
Resources

- Chaitanya K. Rudrabhatla, “Comparison of Event Choreography and Orchestration Techniques in Microservice Architecture” Int. Journal of Advanced Computer Science and Applications(IJACSA), 9(8), 2018. http://dx.doi.org/10.14569/IJACSA.2018.090804 https://thesai.org/Publications/ViewPaper?Volume=9&Issue=8&Code=IJACSA&SerialNo=4

- Chaitanya K. Rudrabhatla, “Impacts of Decomposition Techniques on Performance and Latency of Microservices” Int. Journal of Advanced Computer Science and Applications(IJACSA), 11(8), 2020. http://dx.doi.org/10.14569/IJACSA.2020.0110803 https://thesai.org/Publications/ViewPaper?Volume=11&Issue=8&Code=IJACSA&SerialNo=3

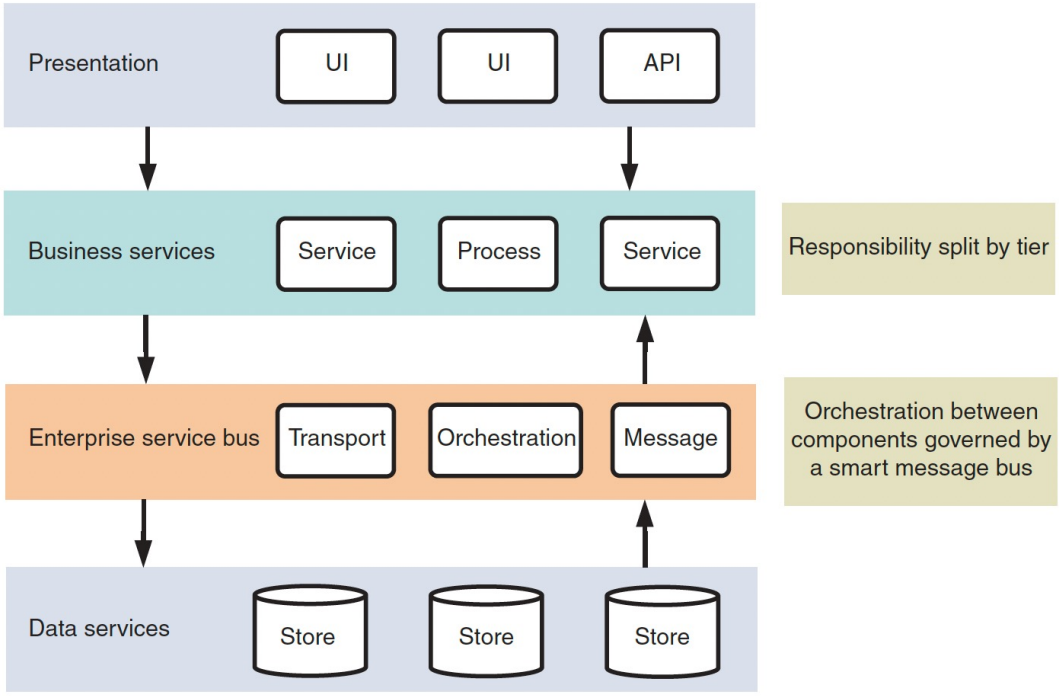
- https://github.com/SebastienBtr/Micro-express

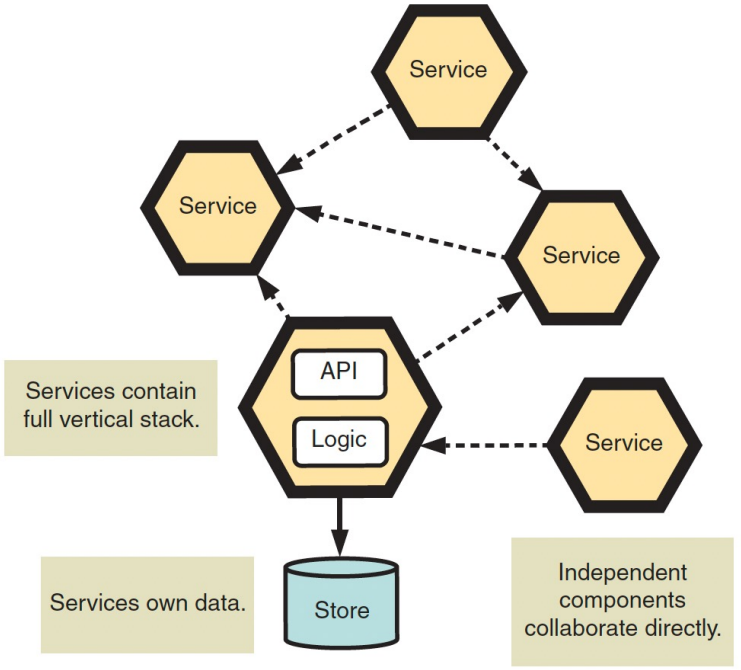
Monoliths, SOA & microservices illustrated



Source: IBM

SOA & microservices illustrated



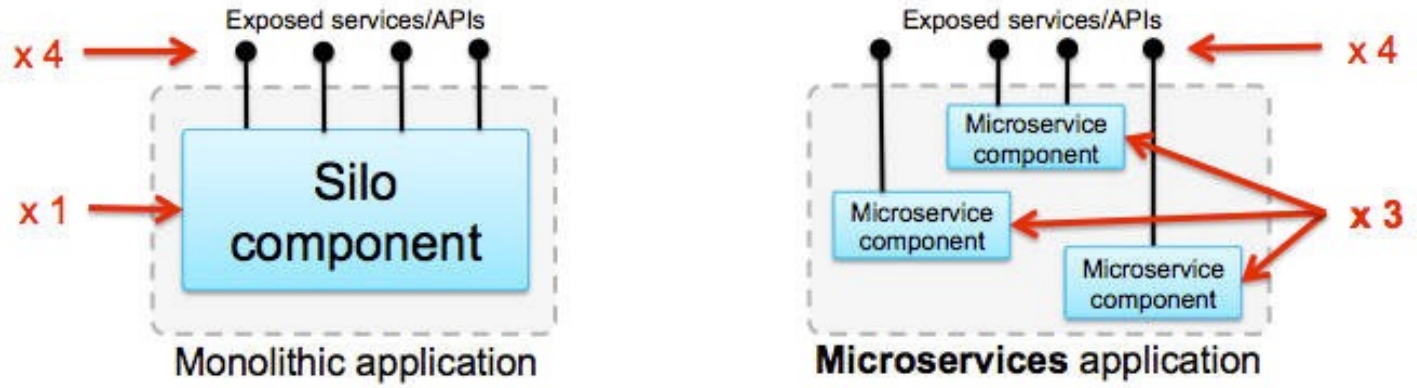


Source: Microservices in Action, M.Bruce, A.Pereira

SOA *and* microservices

Instead of “SOA vs microservices”

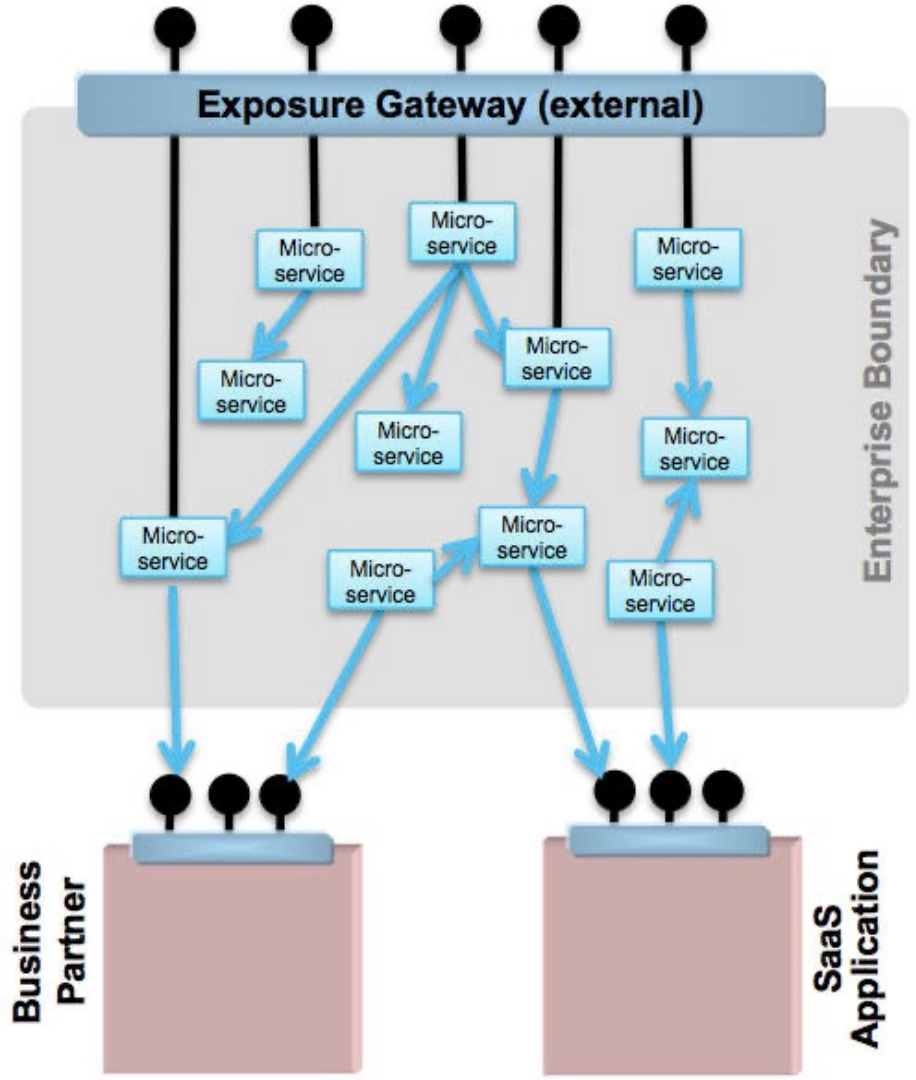
- SOA is about exposing functionality to the outside
- Microservices is about **internally** implementing functionality



Source: IBM

SOA and microservices

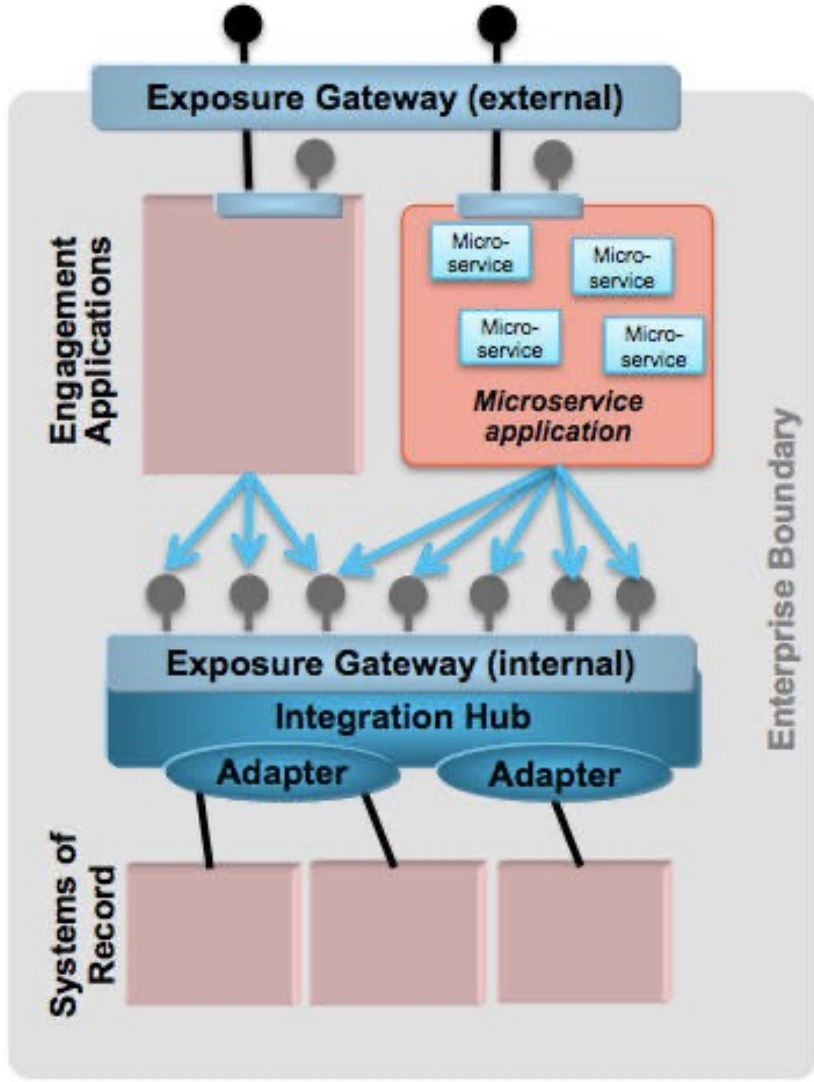
Clean development
(no legacy apps)



Source: IBM

SOA and microservices

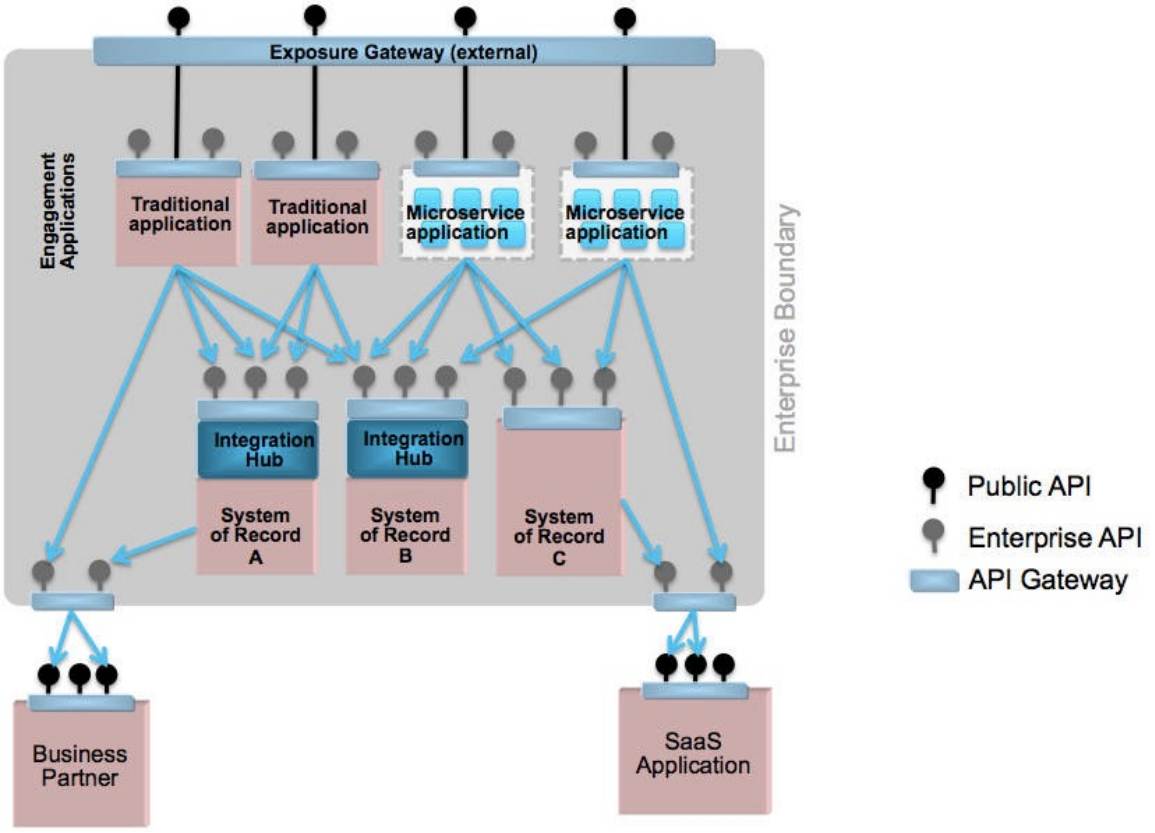
Micoservices need to co-exist with existing apps



Source: IBM

SOA, microservices and APIs

Need to create integration hubs



Source: IBM

Παράδειγμα 1 SOA / microservices

[Θέμα κανονικής εξέτασης SaaS 2020-2021, & εργασίας ΤΛ 2021-2022]

Διαλειτουργικότητα διοδίων ως SaaS

- Τα ακόλουθα ηλεκτρονικά συστήματα διέλευσης διοδίων διαλειτουργούν (δηλαδή επιτρέπουν την χρήση όλων των πομποδεκτών σε όλα τα δίκτυα): aodos.gr, gefyra.gr, egnatia.eu, kentrikiodos.gr, moreas.com.gr, neaodos.gr, olympiaodos.gr.

Παράδειγμα 1 SOA / microservices

- Υποθέτουμε τα εξής (που δεν απέχουν και πολύ από την πραγματικότητα):
  - δεν χρησιμοποιούν όλα την ίδια εφαρμογή διαχείρισης διελεύσεων.
  - όλα υλοποιούν, το καθένα με δικό του τρόπο, τις λειτουργίες "provider" (δήλωση σε ποιον πάροχο ανήκει ένας πομποδέκτης), "load" (φόρτωση πομποδέκτη με χρήματα), "pass" (διέλευση) και "balance" (υπόλοιπο χρημάτων στον πομποδέκτη).

Παράδειγμα 1 SOA / microservices

Θέλουμε να παρέχουμε ως SaaS την υπηρεσία διαλειτουργικότητας.

- Προτείνετε μια αρχιτεκτονική για την εφαρμογή μας, που να βασίζεται στις αρχές SOA (ΑΜ σε άρτιο αριθμό) ή στις αρχές microservices (ΑΜ σε περιττό αριθμό)
- Παραδώστε ένα διάγραμμα UML component με τα στοιχεία της εφαρμογής μας, συνοδευόμενο από πολύ σύντομο σχολιασμό.

Παράδειγμα 2 SOA / Microservices

Ζωντανά διαγράμματα αγορών ενέργειας

- Το “EnergyLive2022” είναι μια δικτυακή εφαρμογή - υπηρεσία που προσφέρει τη δυνατότητα οι χρήστες να παρακολουθούν τιμές μεγεθών της αγοράς ηλεκτρικής ενέργειας στην Ευρώπη μέσω του φυλλομετρητή τους. Τα δεδομένα συλλέγονται αυτόματα από την εφαρμογή και εμφανίζονται στην ιστοσελίδα της.

Παράδειγμα 2 SOA / Microservices

Θέλουμε να παρέχουμε ως SaaS τη θέαση των δεδομένων αγορών ενέργειας

- Προτείνετε μια αρχιτεκτονική microservices για την εφαρμογή μας.
- Παραδώστε ένα διάγραμμα UML component με τα στοιχεία της εφαρμογής μας, συνοδευόμενο από πολύ σύντομο σχολιασμό.